

助教模擬考期中複習

Problem 1, 2, 3, 4

題目目錄

小四數學(乘積最大化)	2
題目敘述	2
題解說明	2
完整程式碼	2
UVa 10019 - Funny Encryption Method	4
題目敘述	4
題解說明	4
完整程式碼	4
UVa 674 - Coin Change	6
題目敘述	6
題解說明	6
完整程式碼	6
UVa 10407 - Simple division	8
題目敘述	8
題解說明	8
完整程式碼	8

小四數學(乘積最大化)

<https://puzzling.stackexchange.com/questions/46521/maximum-result-with-digits>

<https://zerojudge.tw/ShowProblem?problemid=a825>

題目敘述

給定 m 個數字 (0-9)，將其全部使用並拆成兩個整數 a 與 b ，求 $a \times b$ 的最大可能值。

題解說明

這是一個貪心演算法問題。為了讓乘積最大，我們必須讓兩個數字的位元長度盡可能接近，且高位數的數字越大越好。

1. 策略邏輯：

- 將所有數字由大到小排序。
- 取前兩個最大的數字分別作為 a 與 b 的最高位。
- 每次將新數字接在當前數值較小的那個整數後面。

2. 數學直觀：若 $a + b$ 為定值，則當 $|a - b|$ 越小時， $a \times b$ 的值越大。

分配過程視覺化 (以 a, b 的增長為例)：

6	3	1	...	數值 a
↓	↑ ↘	↑ ↘		
5	→ 4	2	...	數值 b

邏輯推導 (以 {6, 5, 4, 3, 2, 1} 為例)：

1. 排序後得 {6, 5, 4, 3, 2, 1}。
2. 初始分配： $a = 6, b = 5$ 。
3. 加入 4：因為 $5 < 6$ ，故 $\rightarrow b = 54$ 。
4. 加入 3：因為 $6 < 54$ ，故 $\rightarrow a = 63$ 。
5. 加入 2：因為 $54 < 63$ ，故 $\rightarrow b = 542$ 。
6. 加入 1：因為 $63 < 542$ ，故 $\rightarrow a = 631$ 。

最終計算：

$$631 \times 542 = 342002$$

完整程式碼

```
1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
4 using namespace std;
5
6 int main()
7 {
8     int m;  cin >> m;
9     vector<int> v(m);
10    for (int i = 0; i < m; ++i) { cin >> v[i]; }
11
12    sort(v.begin(), v.end(), greater<int>());
13    int a = v[0];
```

```
14     int b = v[1];
15     for (int i = 2; i < m; ++i)
16     {
17         if (a <= b)
18         {
19             a *= 10;
20             a += v[i];
21         }
22         else
23         {
24             b *= 10;
25             b += v[i];
26         }
27     }
28
29     cout << a * b << "\n";
30
31     return 0;
32 }
```

UVa 10019 - Funny Encryption Method

<https://zerojudge.tw/ShowProblem?problemid=e545>

<https://vjudge.net/problem/UVA-10019>

題目敘述

給定正整數 N ，計算兩個位元數值：

- b_1 ：十進制轉二進制後 1 的個數。
- b_2 ：十六進制轉二進制後 1 的個數。

題解說明

本題重點在於 b_2 ：將輸入的數字字面上視為十六進制。例如輸入 265 就看作 $265_{(16)}$ ，而非先轉成十六進制。

舉例說明：

若 $N = 265$ ：

- b_1 ： $265_{(10)} = 100001001_2$ （共 3 個 1）。
- b_2 ：將 2, 6, 5 拆開分別轉二進制：

$$2 \rightarrow 0010, 6 \rightarrow 0110, 5 \rightarrow 0101$$

總計 $1 + 2 + 2 = 5$ 個 1。

完整程式碼

```
1 #include <iostream>
2 using namespace std;
3
4 int Dec_to_Bin(int n)
5 {
6     int b1 = 0;
7     while (n >= 1)
8     {
9         if (n % 2 == 1) { b1++; }
10        n /= 2;
11    }
12
13    return b1;
14 }
15
16 int Hex_to_Bin(int n)
17 {
18     int b2 = 0;
19     while (n >= 1)
20     {
21         b2 += Dec_to_Bin(n % 10);
22         n /= 10;
23     }
24
25    return b2;
26 }
27
28 int main()
29 {
30     int T; cin >> T;
31     while (T--)
32     {
```

```
33     int N; cin >> N;
34     cout << Dec_to_Bin(N) << " " << Hex_to_Bin(N) << "\n";
35 }
36
37 return 0;
38 }
```

UVa 674 - Coin Change

<https://zerojudge.tw/ShowProblem?problemid=d253>
<https://vjudge.net/problem/UVa-674>

題目敘述

給定五種面額的硬幣 (1, 5, 10, 25, 50)，求湊出金額 N 的所有組合數。

題解說明

本題屬於經典的「組合找零」問題。我們定義 $dp[j]$ 為組成金額 j 的方法數。為了節省空間，我們使用一維陣列進行迭代更新：

1. 狀態轉移邏輯：對於每一種面額 c_i ，我們更新所有的金額 j ($j \geq c_i$)：

$$dp[j]^{new} = dp[j]^{old} + dp[j - c_i]^{new}$$

- $dp[j]^{old}$ ：不使用面額 c_i 時，組成金額 j 的方法數。
 - $dp[j - c_i]^{new}$ ：使用面額 c_i 後，剩餘金額 $j - c_i$ 的方法數。
2. 避免重複計算：由於我們是「逐一加入硬幣種類」而非逐一填充金額，這確保了我們計算的是**組合 (Combination)** 而非排列 (Permutation)。
3. 邊界條件：

$$dp[0] = 1$$

動態規劃填表過程 (以金額 0-26 為例)：

金額	初始	+1 元	+5 元	+10 元	+25 元
0	1	1	1	1	1
5	0	1	2	2	2
10	0	1	3	4	4
25	0	1	6	12	13
26	0	1	6	12	13

註：表格展現了當硬幣種類逐一加入時，各個金額方法數的演變過程。

完整程式碼

```
1 #include <iostream>
2 #include <array>
3 using namespace std;
4
5 int const MAX = 7490;
6
7 int main()
8 {
9     array<int, MAX> dp = {0};
10    array<int, 5> cent = {1, 5, 10, 25, 50};
11
12    dp[0] = 1;
13    for (int i = 0; i < 5; ++i)
14    {
15        for (int j = cent[i]; j < MAX; ++j)
```

```
16         {
17             dp[j] = dp[j] + dp[j - cent[i]];
18         }
19     }
20
21     int N;
22     while (cin >> N) { cout << dp[N] << "\n"; }
23
24     return 0;
25 }
```

UVa 10407 - Simple division

<https://zerojudge.tw/ShowProblem?problemid=e565>

<https://vjudge.net/problem/UVa-10407>

題目敘述

給定一組整數數列 $\{n_1, n_2, \dots, n_k\}$ ，求一個最大的正整數 d ，使得數列中所有數字除以 d 後的餘數皆相同。

題解說明

這是一題關於 **同餘性質** 的題目。若 a 與 b 除以 d 的餘數相同，則：

$$a \equiv b \pmod{d} \Rightarrow (a - b) \equiv 0 \pmod{d}$$

這代表 d 必須是 $(a - b)$ 的因數。

1. 解題策略：

- 假設餘數為 r ，則每個數可以表示為 $n_i = q_i \times d + r$ 。
- 觀察相鄰兩項的差： $n_{i+1} - n_i = (q_{i+1} - q_i) \times d$ 。
- 由此可知， d 必定是所有差值 $(n_{i+1} - n_i)$ 的公因數。
- 為了求出最大的 d ，我們對所有相鄰項的差值取 **最大公因數** (“GCD”)。

2. 實作要點：

- 持續讀入整數直到遇到 0 結束該筆測試資料。
- 計算數列中每兩個相鄰數字的差值，並取絕對值 $|n_i - n_{i-1}|$ 。
- 使用**輾轉相除法**對所有差值求 “GCD”，所得結果即為最大可能的 d 。

推導範例：

輸入數列：701, 1059, 1417, 2312, 0

- 計算相鄰項差值：
 - $|1059 - 701| = 358$
 - $|1417 - 1059| = 358$
 - $|2312 - 1417| = 895$
- 求最大公因數： $\gcd(358, 358, 895) = 179$

驗算 (餘數皆需相同)：

- $701 = 3 \times 179 + 164$
- $1059 = 5 \times 179 + 164$
- $1417 = 7 \times 179 + 164$
- $2312 = 12 \times 179 + 164$

(餘數皆為 164，故最大正整數 $d = 179$)

完整程式碼

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int GCD(int a, int b)
6 {
7     if (a == 0) { return b; }
8     if (b == 0) { return a; }
9
10    return GCD (b, a % b);
11 }
12
```

```
13 int main()
14 {
15     int first;
16     while (cin >> first and first != 0)
17     {
18         vector<int> A; A.push_back(first);
19
20         int n;
21         while (cin >> n and n != 0) { A.push_back(n); }
22
23         vector<int> B;
24         for (int i = 1; i < A.size(); ++i) { B.push_back(abs(A[i] - A[i-1])); }
25
26         int ans = B[0];
27         for (int i = 1; i < B.size(); ++i) { ans = GCD(B[i], ans); }
28
29         cout << ans << "\n";
30     }
31
32     return 0;
33 }
34
```